

Assignment 1 - PostgreSQL Basics

▪ General Guidelines

1. Use snake_case naming convention for all database objects.
2. Primary keys must be UUID.
3. Use PostgreSQL SQL syntax.
4. Follow proper auditing practices.

▪ Instruction Executed

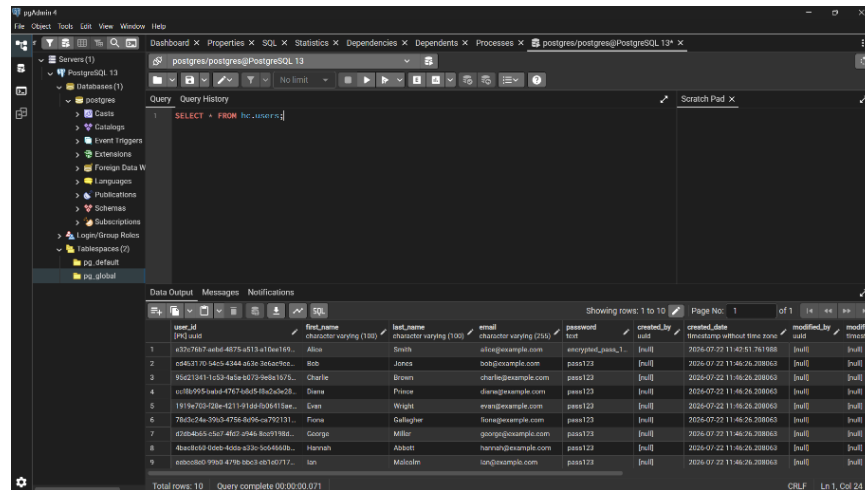
1. CREATE DATABASE homecare_db;
2. CREATE SCHEMA hc;
3. CREATE TABLE hc.users (
user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
first_name VARCHAR(100),
last_name VARCHAR(100),
email VARCHAR(255) UNIQUE,
password TEXT,
created_by UUID,
created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
modified_by UUID,
modified_date TIMESTAMP,
is_active BOOLEAN DEFAULT TRUE,
is_deleted BOOLEAN DEFAULT FALSE,
birth_date DATE,
address TEXT,
mobile_number VARCHAR(20)
);
4. SELECT * FROM hc.users;
5. INSERT INTO hc.users (
first_name,
last_name,
email,
password,
birth_date,
address,
mobile_number
) VALUES (
'Alice',
'Smith',
'alice@example.com',
'encrypted_pass_123',
'1985-06-15',
'123 Maple Street',
'555-1234'
);

Work done by: Mitesh Bhagat

6. INSERT INTO hc.users (first_name, last_name, email, password, birth_date, address, mobile_number)

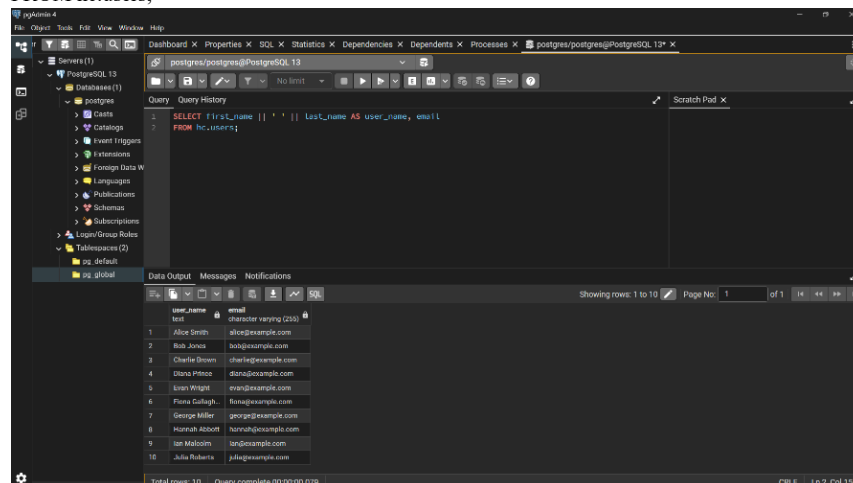
VALUES

('Bob', 'Jones', 'bob@example.com', 'pass123', '1990-08-20', '456 Oak Ave', '555-5678'),
('Charlie', 'Brown', 'charlie@example.com', 'pass123', '1975-12-05', '789 Pine Rd', '555-9012'),
('Diana', 'Prince', 'diana@example.com', 'pass123', '1988-03-22', '321 Elm St', '555-3456'),
('Evan', 'Wright', 'evan@example.com', 'pass123', '1995-11-10', '654 Birch Blvd', '555-7890'),
('Fiona', 'Gallagher', 'fiona@example.com', 'pass123', '1992-07-08', '987 Cedar Ln', '555-2345'),
('George', 'Miller', 'george@example.com', 'pass123', '1980-02-14', '147 Spruce Ct', '555-6789'),
('Hannah', 'Abbott', 'hannah@example.com', 'pass123', '1998-09-30', '258 Willow Dr', '555-0123'),
('Ian', 'Malcolm', 'ian@example.com', 'pass123', '1970-05-18', '369 Aspen Way', '555-4567'),
('Julia', 'Roberts', 'julia@example.com', 'pass123', '1982-10-25', '741 Cherry Pl', '555-8901');



user_id	first_name	last_name	email	password	created_by	created_date	modified_by
1	Bob	Jones	bob@example.com	pass123	[null]	2024-07-22 11:40:51.761888	[null]
2	Charlie	Brown	charlie@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
3	Diana	Prince	diana@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
4	Evan	Wright	evan@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
5	Fiona	Gallagher	fiona@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
6	George	Miller	george@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
7	Hannah	Abbott	hannah@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
8	Ian	Malcolm	ian@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
9	Julia	Roberts	julia@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
10							

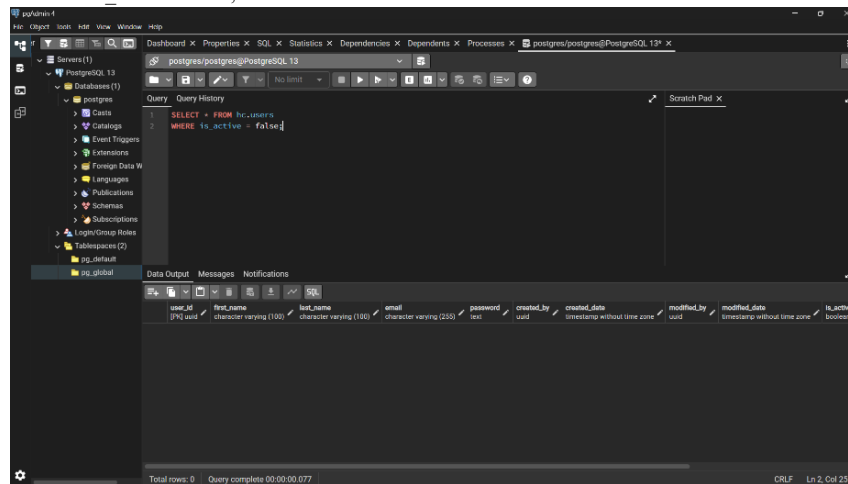
7. SELECT first_name || ' ' || last_name AS user_name, email
FROM hc.users;



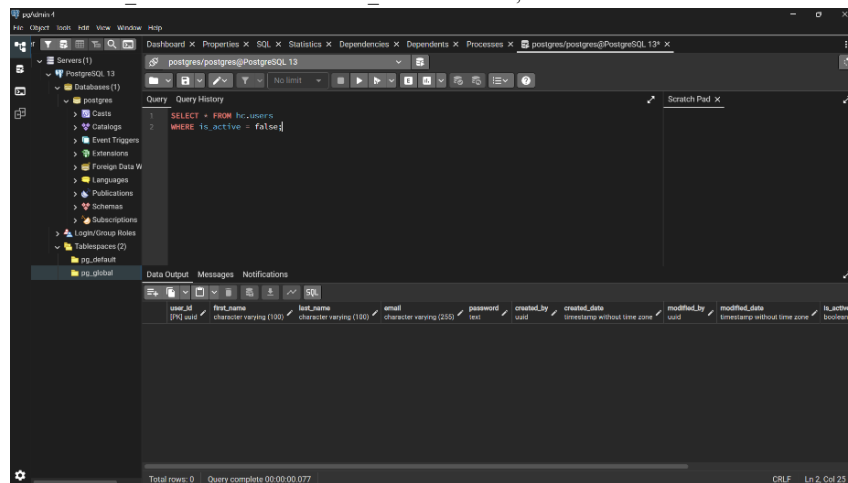
user_name	email
Bob Jones	bob@example.com
Charlie Brown	charlie@example.com
Diana Prince	diana@example.com
Evan Wright	evan@example.com
Fiona Gallagher	fiona@example.com
George Miller	george@example.com
Hannah Abbott	hannah@example.com
Ian Malcolm	ian@example.com
Julia Roberts	julia@example.com

Work done by: Mitesh Bhagat

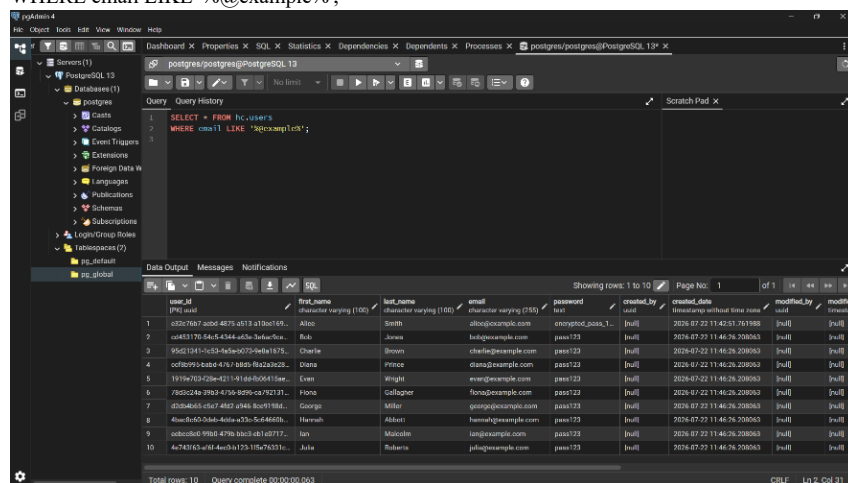
8. `SELECT * FROM hc.users`
`WHERE is_active = false;`



9. `SELECT * FROM hc.users`
`WHERE first_name LIKE 'A%' AND last_name LIKE '%i';`

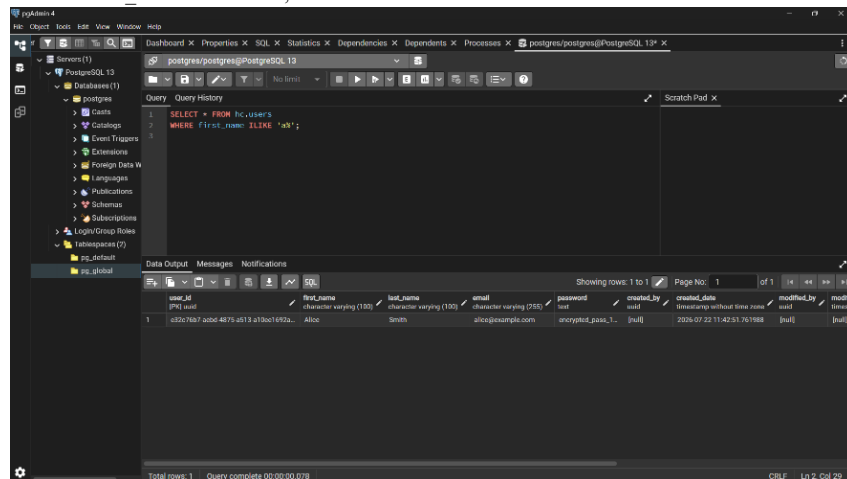


10. `SELECT * FROM hc.users`
`WHERE email LIKE '%@example%';`



Work done by: Mitesh Bhagat

11. SELECT * FROM hc.users
WHERE first_name ILIKE 'a%';



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

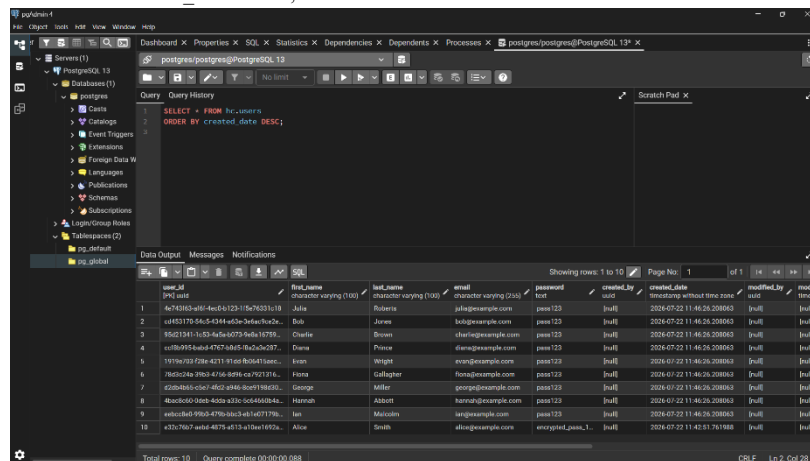
```
1 SELECT * FROM hc.users
2 WHERE first_name ILIKE 'a%';
```

The Data Output pane shows the results of the query. The table has 11 columns: user_id, first_name, last_name, email, password, created_by, created_date, and modified_by. The results are as follows:

user_id	first_name	last_name	email	password	created_by	created_date	modified_by
1	alice	smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]

Total rows: 1 Query complete 00:00:00.079

12. SELECT * FROM hc.users
ORDER BY created_date DESC;



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

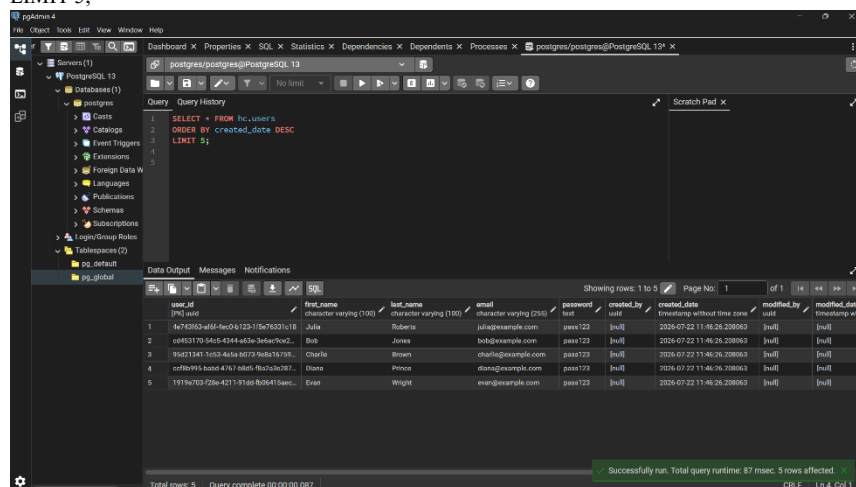
```
1 SELECT * FROM hc.users
2 ORDER BY created_date DESC;
```

The Data Output pane shows the results of the query. The table has 11 columns: user_id, first_name, last_name, email, password, created_by, created_date, and modified_by. The results are as follows:

user_id	first_name	last_name	email	password	created_by	created_date	modified_by
1	alice	smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]
2	bob	jones	bob@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
3	charlie	brown	charlie@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
4	diana	prince	diana@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
5	evan	wright	evan@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
6	frank	gallagher	frank@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
7	george	miller	george@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
8	hannah	abbot	hannah@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
9	ian	malcolm	ian@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
10	alice	smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]

Total rows: 10 Query complete 00:00:00.088

13. SELECT * FROM hc.users
ORDER BY created_date DESC
LIMIT 5;



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 SELECT * FROM hc.users
2 ORDER BY created_date DESC
3 LIMIT 5;
```

The Data Output pane shows the results of the query. The table has 11 columns: user_id, first_name, last_name, email, password, created_by, created_date, and modified_by. The results are as follows:

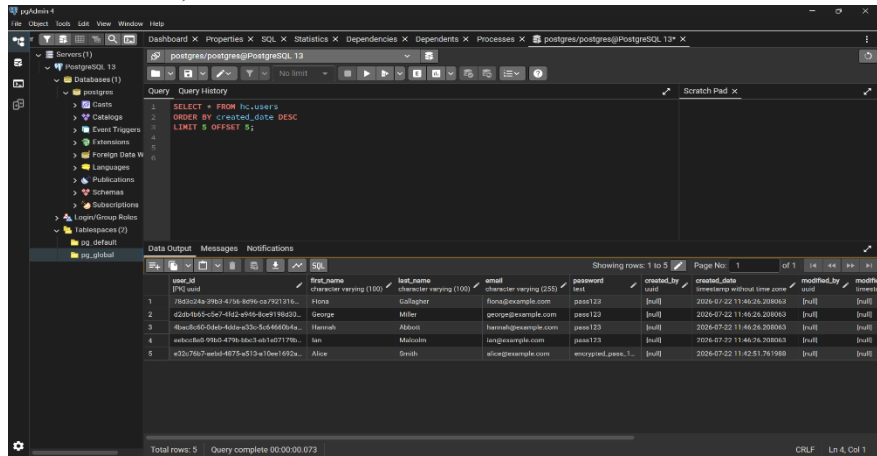
user_id	first_name	last_name	email	password	created_by	created_date	modified_by
1	alice	smith	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.761988	[null]
2	bob	jones	bob@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
3	charlie	brown	charlie@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
4	diana	prince	diana@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]
5	evan	wright	evan@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]

Total rows: 5 Query complete 00:00:00.087

Successfully run. Total query runtime: 87 msec. 5 rows affected.

Work done by: Mitesh Bhagat

14. `SELECT * FROM hc.users`
`ORDER BY created_date DESC`
`LIMIT 5 OFFSET 5;`



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'postgres@PostgreSQL 13'. The central pane shows a SQL query: `SELECT * FROM hc.users ORDER BY created_date DESC LIMIT 5 OFFSET 5;`. The bottom pane displays the query results in a table format, showing 5 rows of user data.

user_id	first_name	last_name	email	password	created_by	created_date	modified_by	modified_date
1	John	Doe	john.doe@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
2	George	Miller	george@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
3	Hannah	Abbot	hannah@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
4	Ian	Maldin	ian@example.com	pass123	[null]	2024-07-22 11:46:26.208063	[null]	[null]
5	Alice	Bradt	alice@example.com	encrypted_pass_1...	[null]	2024-07-22 11:42:51.781980	[null]	[null]

Total rows: 5 Query complete 00:00:00.072 ORL Ln: 4, Col: 1

Assignment 2 - PostgreSQL Basics

General Instructions

- Use PostgreSQL syntax only
- Follow snake_case naming convention
- Do not include query outputs
- Focus on correct SQL logic
- Database: training_db
- Schema: tdb

Available Tables

- users
 - roles
 - service_types
 - categories
 - sub_categories
- Instruction Executed

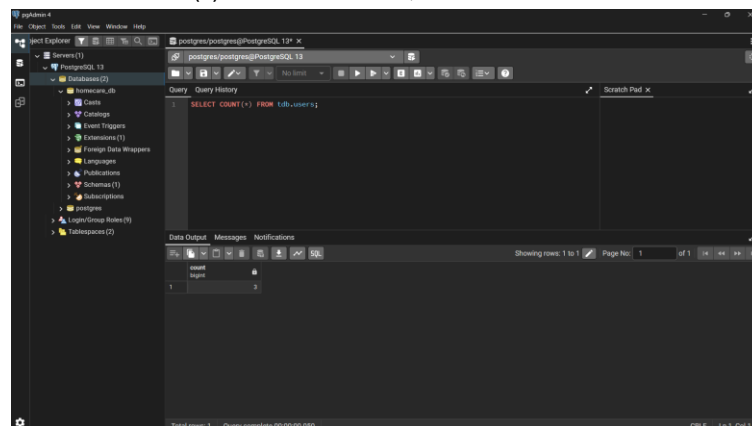
Instruction Executed

1. Importing form Folder

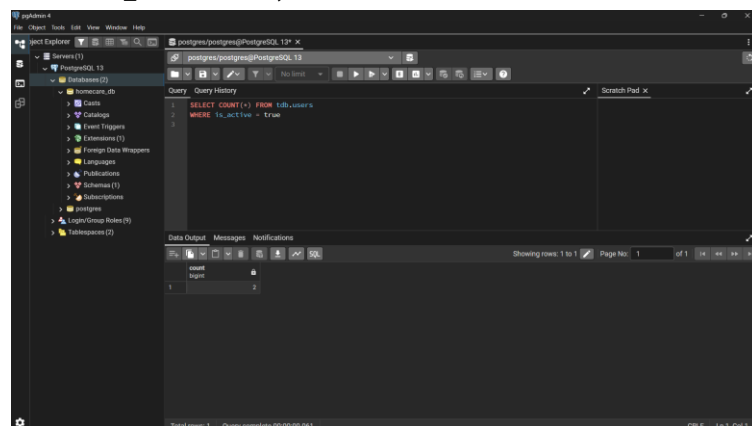
1. File Name: "PostgreSQL Assignment 1"

2. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

1. SELECT COUNT(*) FROM tdb.users;

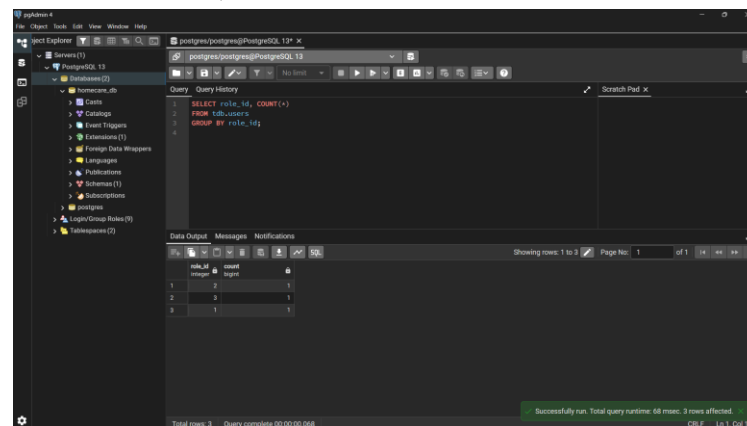


2. SELECT COUNT(*) FROM tdb.users
WHERE is_active = true;



Work done by: Mitesh Bhagat

3. `SELECT role_id, COUNT(*)`
`FROM tdb.users`
`GROUP BY role_id;`



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

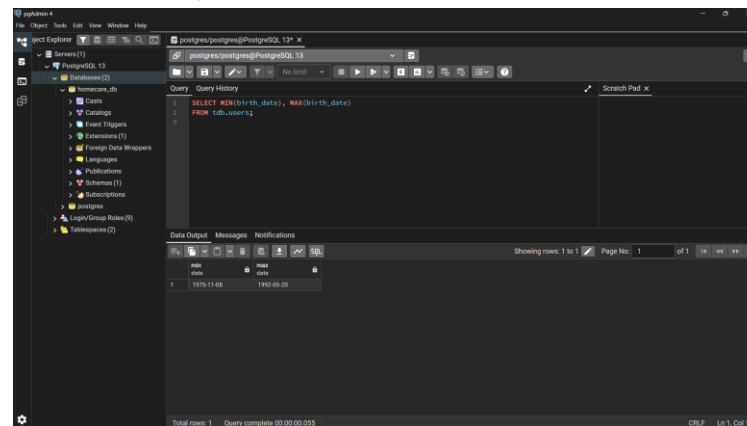
```
1 SELECT role_id, COUNT(*)
2 FROM tdb.users;
3 GROUP BY role_id;
```

The Data Output pane shows the results of the query:

role_id	count
1	3
2	1
3	1

At the bottom, a status bar indicates: "Successfully ran. Total query runtime: 68 msec. 3 rows affected. CRLF Ln 1, Col 1".

4. `SELECT MIN(birth_date), MAX(birth_date)`
`FROM tdb.users;`



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

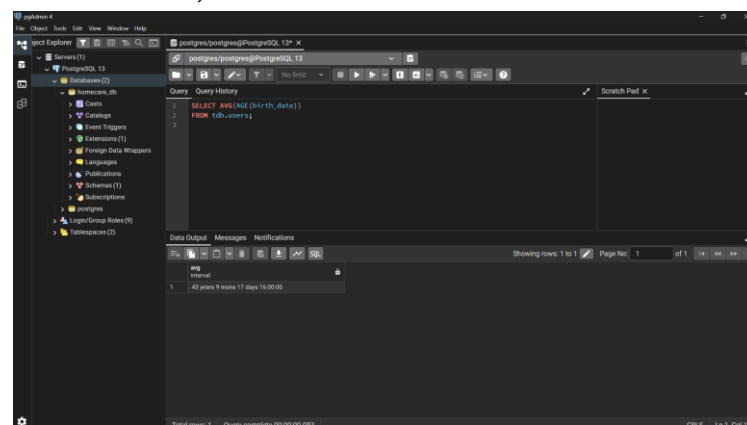
```
1 SELECT MIN(birth_date), MAX(birth_date)
2 FROM tdb.users;
```

The Data Output pane shows the results of the query:

min_date	max_date
1976-11-08	1992-05-20

At the bottom, a status bar indicates: "Total rows: 1 Query complete 00:00:00.055 CRLF Ln 1, Col 1".

5. `SELECT AVG(AGE(birth_date))`
`FROM tdb.users;`



The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL query:

```
1 SELECT AVG(AGE(birth_date))
2 FROM tdb.users;
```

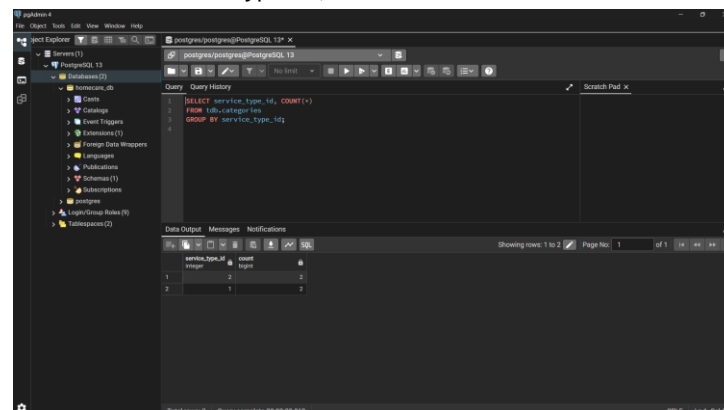
The Data Output pane shows the results of the query:

avg
42 years 9 months 17 days 16:00:00

At the bottom, a status bar indicates: "Total rows: 1 Query complete 00:00:00.053 CRLF Ln 1, Col 1".

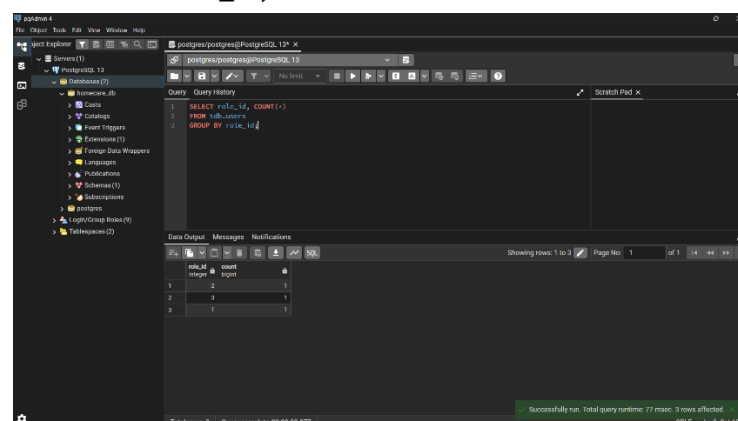
Work done by: Mitesh Bhagat

6. `SELECT service_type_id, COUNT(*)`
`FROM tdb.categories`
`GROUP BY service_type_id;`

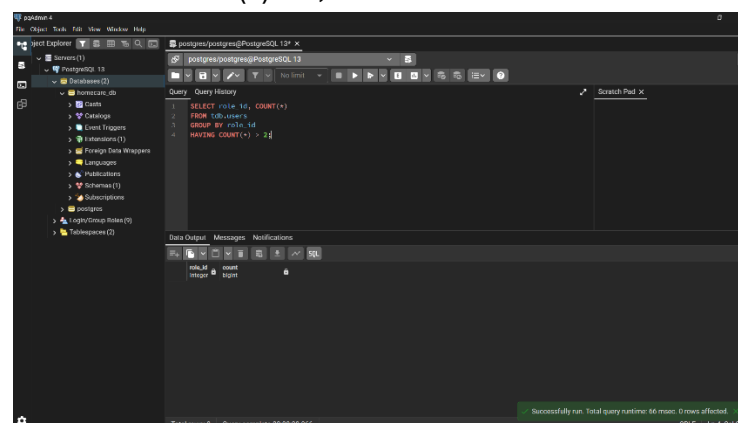


3. GROUP BY & HAVING

1. `SELECT role_id, COUNT(*)`
`FROM tdb.users`
`GROUP BY role_id;`



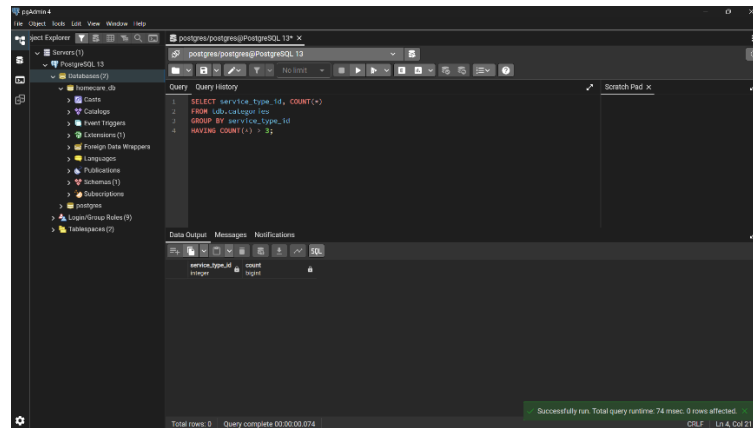
2. `SELECT role_id, COUNT(*)`
`FROM tdb.users`
`GROUP BY role_id`
`HAVING COUNT(*) > 2;`



Work done by: Mitesh Bhagat

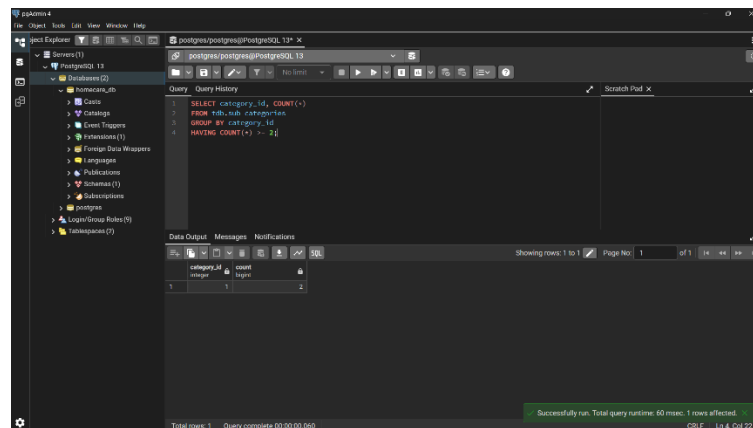
3.

```
SELECT service_type_id, COUNT(*)  
FROM tdb.categories  
GROUP BY service_type_id  
HAVING COUNT(*) > 3;
```



4.

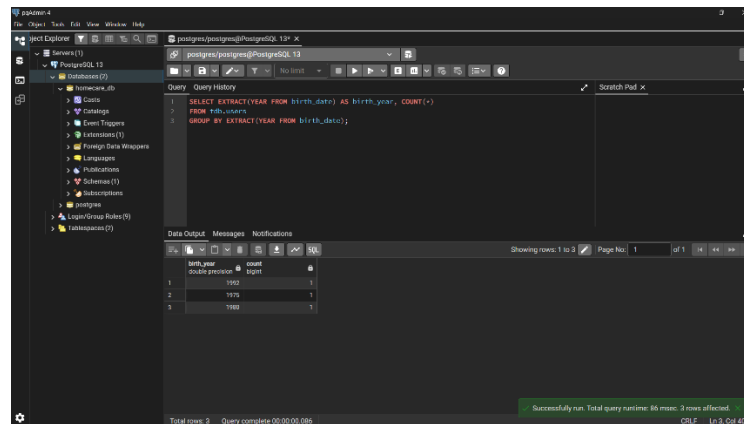
```
SELECT category_id, COUNT(*)  
FROM tdb.sub_categories  
GROUP BY category_id  
HAVING COUNT(*) >= 2;
```



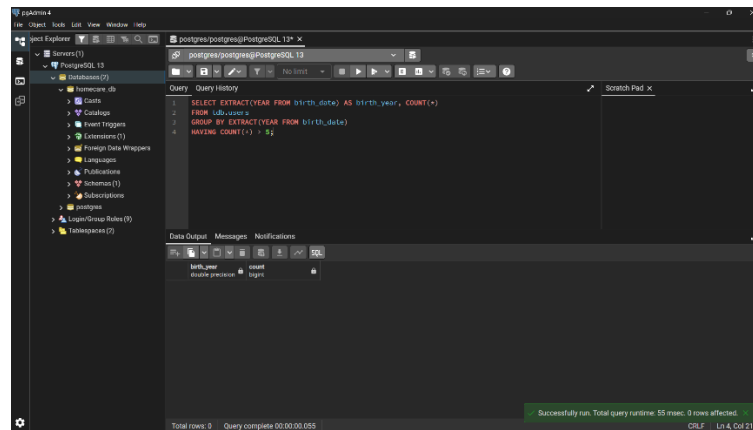
5.

```
SELECT EXTRACT(YEAR FROM birth_date) AS birth_year, COUNT(*)  
FROM tdb.users  
GROUP BY EXTRACT(YEAR FROM birth_date);
```

Work done by: Mitesh Bhagat



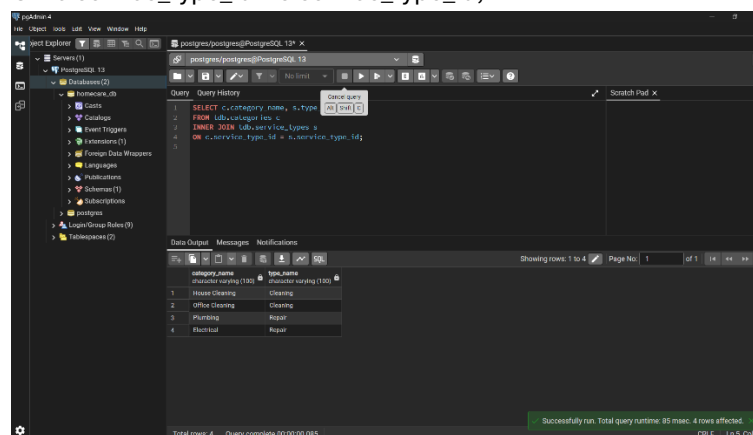
6. `SELECT EXTRACT(YEAR FROM birth_date) AS birth_year, COUNT(*)`
`FROM tdb.users`
`GROUP BY EXTRACT(YEAR FROM birth_date)`
`HAVING COUNT(*) > 5;`



4. Joins

1. **Inner Join: Fetch categories with service type names**

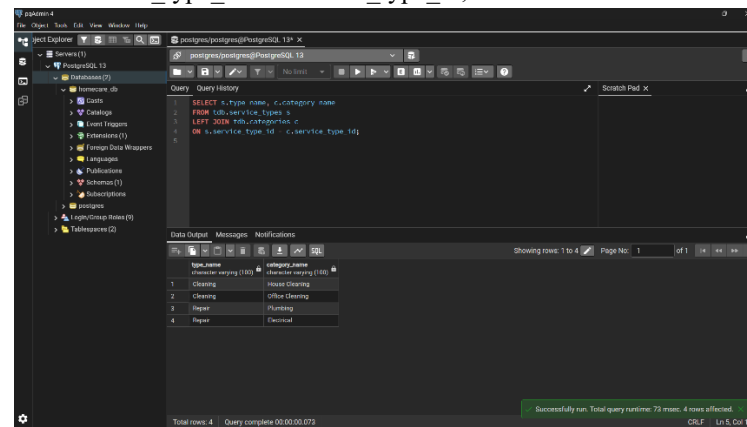
`SELECT c.category_name, s.type_name`
`FROM tdb.categories c`
`INNER JOIN tdb.service_types s`
`ON c.service_type_id = s.service_type_id;`



Work done by: Mitesh Bhagat

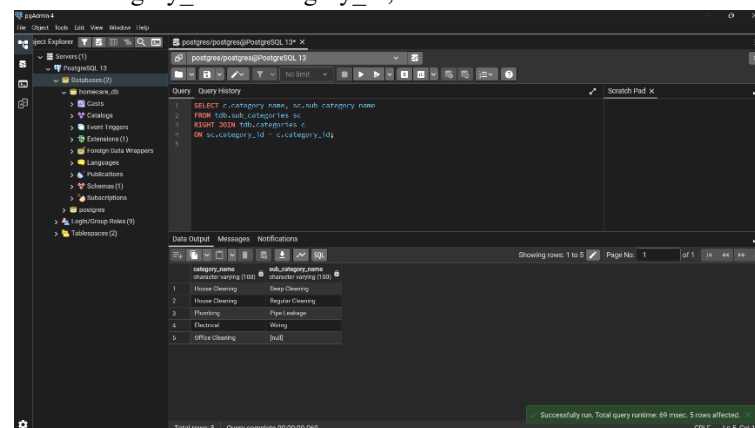
2. **Left Join: Fetch all service types including those without categories**

```
SELECT s.type_name, c.category_name
FROM tdb.service_types s
LEFT JOIN tdb.categories c
ON s.service_type_id = c.service_type_id;
```



3. **Right Join: Fetch all categories even if they have no sub-categories**

```
SELECT c.category_name, sc.sub_category_name
FROM tdb.sub_categories sc
RIGHT JOIN tdb.categories c
ON sc.category_id = c.category_id;
```

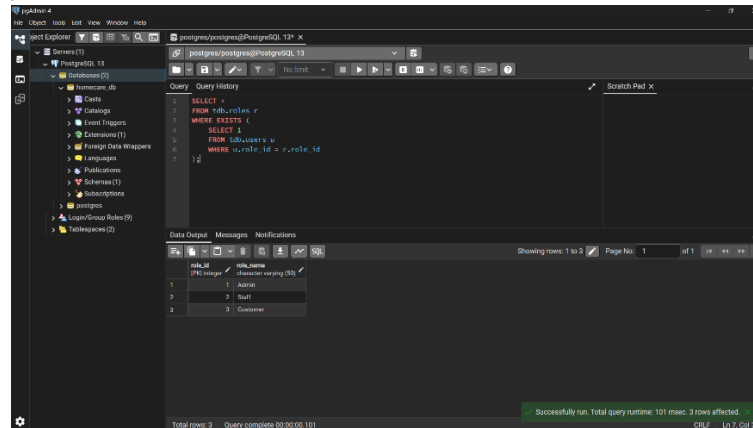


Work done by: Mitesh Bhagat

5. Exists

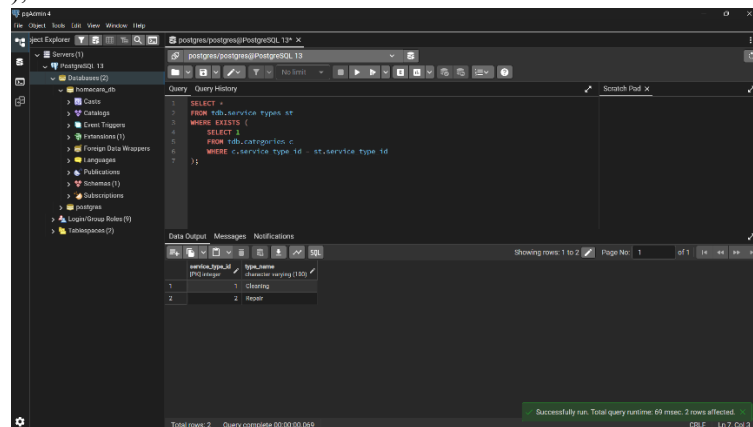
1.

```
SELECT *  
FROM tdb.roles r  
WHERE EXISTS (  
    SELECT 1  
    FROM tdb.users u  
    WHERE u.role_id = r.role_id  
);
```



2.

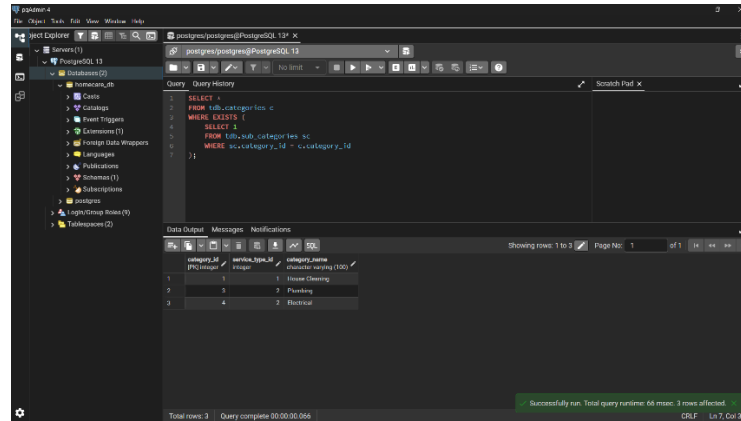
```
SELECT *  
FROM tdb.service_types st  
WHERE EXISTS (  
    SELECT 1  
    FROM tdb.categories c  
    WHERE c.service_type_id = st.service_type_id  
);
```



Work done by: Mitesh Bhagat

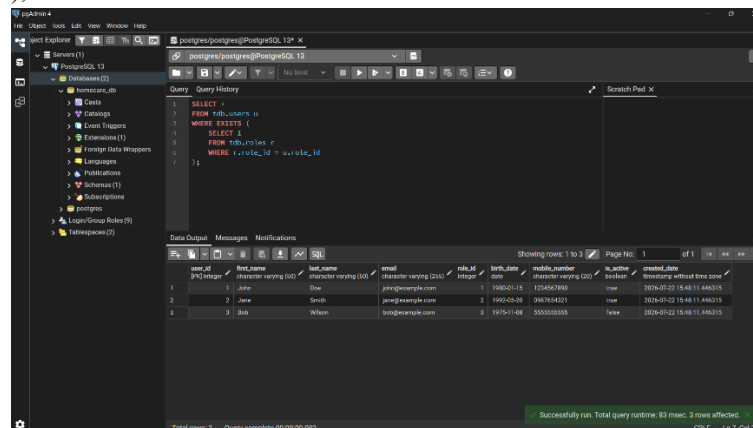
3.

```
SELECT *  
FROM tdb.categories c  
WHERE EXISTS (  
    SELECT 1  
    FROM tdb.sub_categories sc  
    WHERE sc.category_id = c.category_id  
);
```



4.

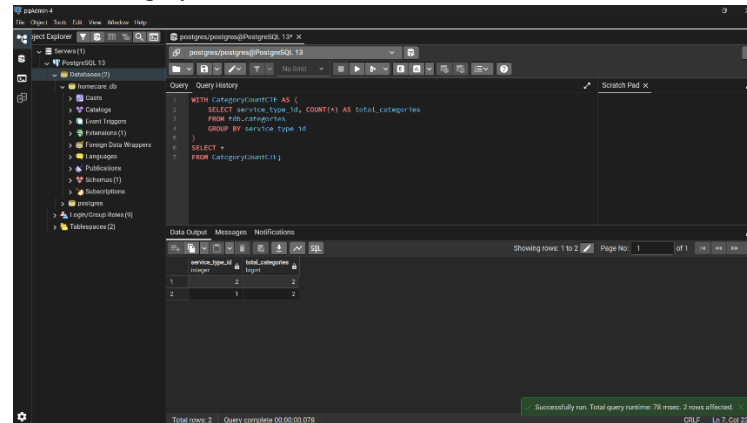
```
SELECT *  
FROM tdb.users u  
WHERE EXISTS (  
    SELECT 1  
    FROM tdb.roles r  
    WHERE r.role_id = u.role_id  
);
```



6. Common Table Expression

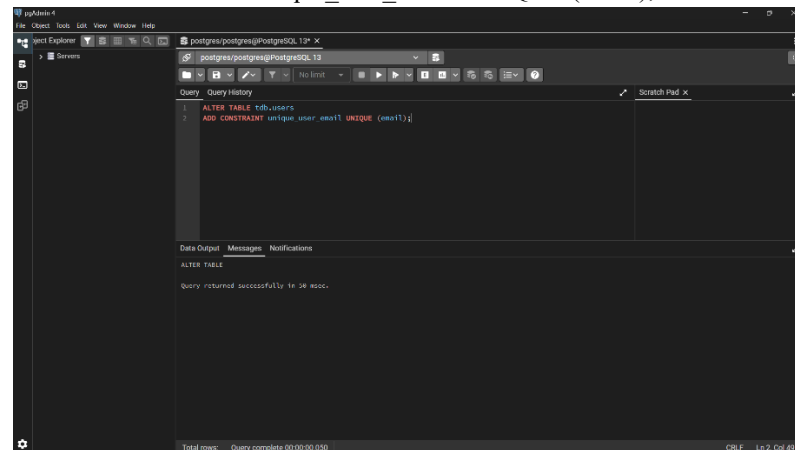
1. Use a CTE to count the number of categories per service type

```
WITH CategoryCountCTE AS (  
    SELECT service_type_id, COUNT(*) AS total_categories  
    FROM tdb.categories  
    GROUP BY service_type_id  
)  
SELECT *  
FROM CategoryCountCTE;
```



7. Constraints

1. ALTER TABLE tdb.users
ADD CONSTRAINT unique_user_email UNIQUE (email);



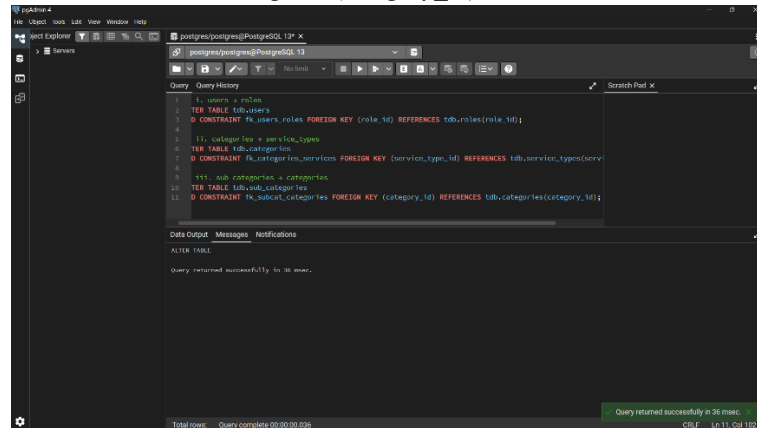
2. -- i. users → roles
ALTER TABLE tdb.users
ADD CONSTRAINT fk_users_roles FOREIGN KEY (role_id) REFERENCES
tdb.roles(role_id);

-- ii. categories → service_types
ALTER TABLE tdb.categories
ADD CONSTRAINT fk_categories_services FOREIGN KEY (service_type_id)
REFERENCES tdb.service_types(service_type_id);

-- iii. sub_categories → categories

Work done by: Mitesh Bhagat

```
ALTER TABLE tdb.sub_categories
ADD CONSTRAINT fk_subcat_categories FOREIGN KEY (category_id)
REFERENCES tdb.categories(category_id);
```

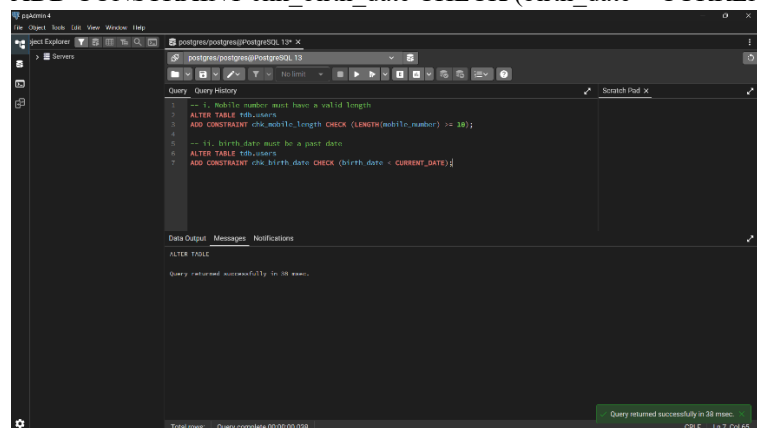


3. -- i. Mobile number must have a valid length

```
ALTER TABLE tdb.users
ADD CONSTRAINT chk_mobile_length CHECK (LENGTH(mobile_number) >=
10);
```

- ii. birth_date must be a past date

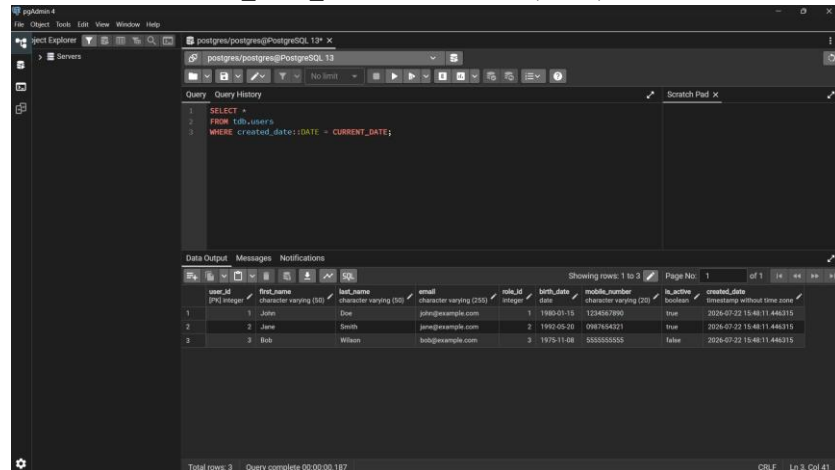
```
ALTER TABLE tdb.users
ADD CONSTRAINT chk_birth_date CHECK (birth_date < CURRENT_DATE);
```



Work done by: Mitesh Bhagat

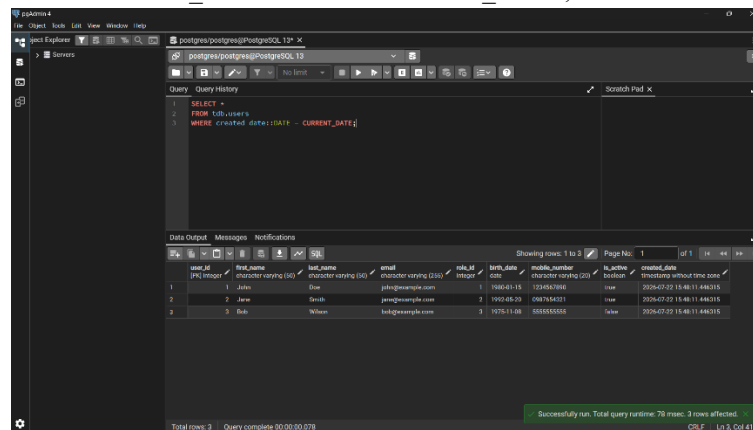
8. Indexes (Create an index on users.email)

1. CREATE INDEX idx_users_email ON tdb.users(email)

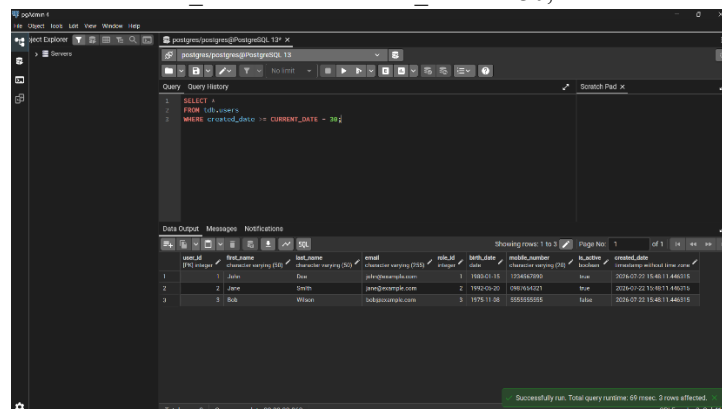


9. Date & Time Handling

1. SELECT *
FROM tdb.users
WHERE created_date::DATE = CURRENT_DATE;

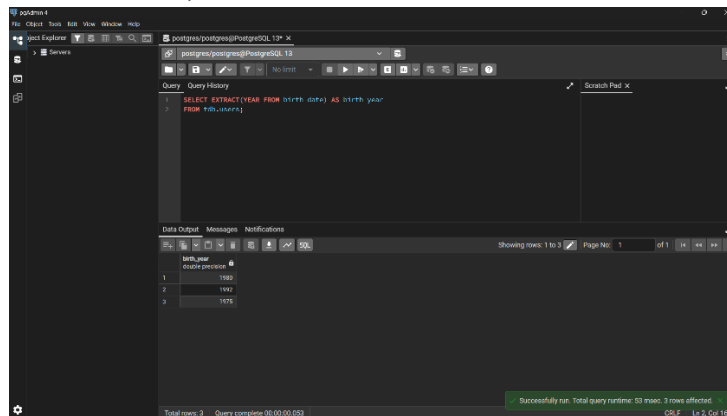


2. SELECT *
FROM tdb.users
WHERE created_date >= CURRENT_DATE - 30;

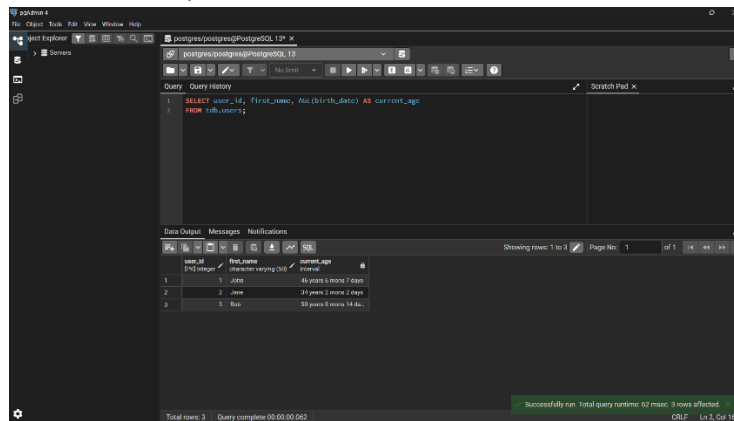


Work done by: Mitesh Bhagat

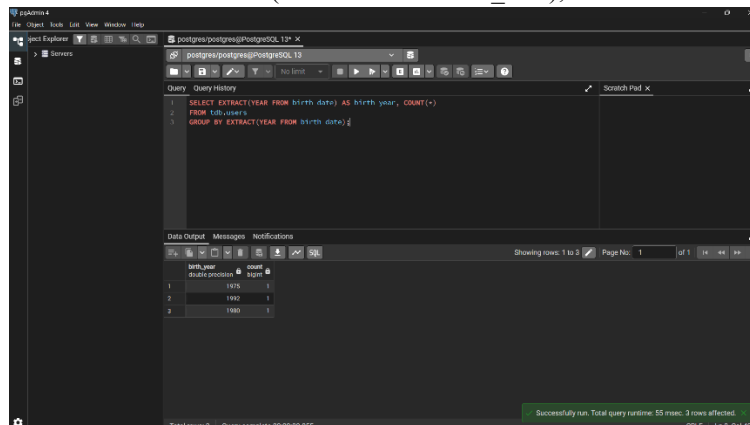
3. `SELECT EXTRACT(YEAR FROM birth_date) AS birth_year
FROM tdb.users;`



4. `SELECT user_id, first_name, AGE(birth_date) AS current_age
FROM tdb.users;`



5. `SELECT EXTRACT(YEAR FROM birth_date) AS birth_year, COUNT(*)
FROM tdb.users
GROUP BY EXTRACT(YEAR FROM birth_date);`



Work done by: Mitesh Bhagat

10. Window Functions (ROW_NUMBER, RANK, LAG, LEAD)

-- a. Assign row numbers to users within each role based on created_date

```
SELECT
    user_id,
    first_name,
    role_id,
    created_date,
    ROW_NUMBER() OVER(PARTITION BY role_id ORDER BY created_date) AS
row_num
FROM tdb.users;
```

-- b. Rank roles based on number of users

```
WITH RoleCounts AS (
    SELECT role_id, COUNT(*) AS total_users
    FROM tdb.users
    GROUP BY role_id
)
SELECT
    role_id,
    total_users,
    RANK() OVER(ORDER BY total_users DESC) AS role_rank
FROM RoleCounts;
```

-- c. Use LAG to fetch previous user creation date per role

```
SELECT
    user_id,
    role_id,
    created_date,
    LAG(created_date) OVER(PARTITION BY role_id ORDER BY created_date) AS
prev_created_date
FROM tdb.users;
```

-- d. Use LEAD to fetch next user creation date per role

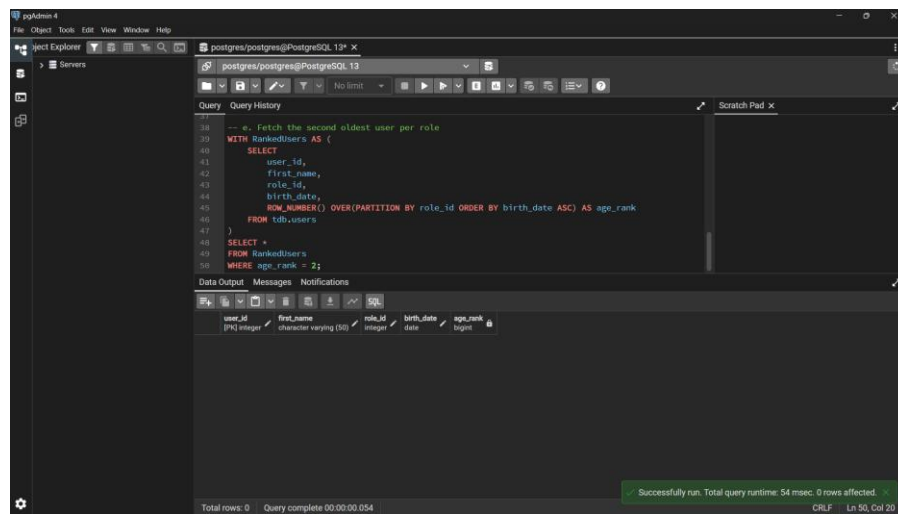
```
SELECT
    user_id,
    role_id,
    created_date,
    LEAD(created_date) OVER(PARTITION BY role_id ORDER BY created_date) AS
next_created_date
FROM tdb.users;
```

-- e. Fetch the second oldest user per role

```
WITH RankedUsers AS (
    SELECT
        user_id,
        first_name,
        role_id,
        birth_date,
        ROW_NUMBER() OVER(PARTITION BY role_id ORDER BY birth_date ASC) AS
age_rank
    FROM tdb.users
)
```

Work done by: Mitesh Bhagat

```
SELECT *  
FROM RankedUsers  
WHERE age_rank = 2;
```



Work done by: Mitesh Bhagat

Assignment 3 - PostgreSQL Basics

- **Instruction Executed**